

Android Malware

Machine Learning Problem Formulation

Introduction: Impact of Android Malware

In The Strait Times article dated Oct 17, 2023 [1], it was reported that from January to August 2023 more than 1,400 victims in Singapore fell prey to malware scams costing \$20.6 million losses. In order to mitigate this problem, Cyber Security Agency of Singapore collaborated with Google to restrict Android users in Singapore to install app only from Google Play store [2].

However, this maybe insufficient:

- Based on the mobile malware threat landscape in 2023 by Kaspersky [3], it is reported that "Attackers continued to use official app marketplaces, such as Google Play, to spread malware. ".
- It was discovered that Malwares in Google Play store was downloaded 600 million times in 2023 [4]. Hence, even restricting app download to official Google Play store alone is not enough to protect Android devices from malware.

Given the above, the ability to detect Android malware becomes quite critical.

Challenges In Detecting Malware

The traditional approach to detect malware based on detecting sequence of bytes or **Signature-based** in the application.

This has been successful in protecting devices in the past yet there are two challenges:

- The traditional approach is based on known malware that is analyzed after the attack. In short it is reactive in nature. With the growth of sheer volume of new malware, this traditional method has struggled to keep up.[5] In Q2 2023 alone, there are 438,962 new malicious mobile malware packages according to Statista report[6].
- In addition new detection evasion technique like **Polymorphic code** to change malware code each time it is launched making signature or even behaviour detection of malware harder to keep up.[7]

Advantage of Machine Learning in Malware Detection

Even after the code change, Malware still call the same set of Android API and require the same set of permission in Android manifest file. Hence, by collecting the API call and permission, we could somehow detect malware. This is not a simple problem though given that benign non-malware Android applications also use the same set of API and permission plus there are a lot of features to process.

This seem to fit nicely with **binary classification** application of machine learning. Our features are set of API or manifest called by the application which is labeled by researchers in this **supervised learning** process and our target is simply **malware vs non-malware (benign)**.

Compared to traditional methods of searching for sugnature, machine learning provides several advantages:

- Machine Learning is able to detect complex pattern of behavior
- Machine Learning is able to detect never seen before malware simply by similarities to behavior of previously known malware. This is able to cope with polymorphic code.
- Machine Learning is able to be retrained with new data and adapted quickly which allow it to cope with malware growth

In conclusion, Machine Learning is quite promising in Malware detection.

References

1. The Strait Times, Oct 17,2023: [Spike in malware scam victims in July, August; losses equivalent to that in first half of 2023](#)
2. The Strait Times, Feb 08, 2024: [Android users in S'pore to be blocked from installing unverified apps as part of anti-scam trial](#)
3. SecureList by Kaspersky: [The mobile malware threat landscape in 2023](#)
4. Kasperksky Nov 09, 2023 [Google Play malware clocks up more than 600 million downloads in 2023](#)
5. ZDNet April 28,2008:[Signature-based antivirus is dead: Get over it](#)
6. Statista : [Number of detected malicious installation packages on mobile devices worldwide from 4th quarter 2015 to 3rd quarter 2023](#)
7. Wikipedia : [Polymorphic Code](#)

DataSet

Our dataset for this project is taken from <https://drebin.mlsec.org/> which is a known dataset of android malware data and described in the papers:

- Daniel Arp, Michael Spreitzenbarth, Malte Hübner, Hugo Gascon, and Konrad Rieck "Drebin: Efficient and Explainable Detection of Android Malware in Your Pocket", 21th Annual Network and Distributed System Security Symposium (NDSS), February 2014
- Michael Spreitzenbarth, Florian Echtler, Thomas Schreck, Felix C. Freling, Johannes Hoffmann, "MobileSandbox: Looking Deeper into Android Applications", 28th International ACM Symposium on Applied Computing (SAC), March 2013

This is a public dataset which we downloaded from

https://figshare.com/articles/dataset/Android_malware_dataset_for_machine_learning_2/58

There are other similar datasets for android malware for example:

- <https://github.com/DefenseDroid/DefenseDroid>
- <https://www.unb.ca/cic/datasets/maldroid-2020.html>
 - file: feature_vectors_syscalls_frequency_5_Cat.csv
 - This is a very comprehensive and clean data set. The problem with this dataset is that it is too clean to demonstrate preprocessing steps in our project.
- <https://www.unb.ca/cic/datasets/andmal2020.html>
 - This data set similar to the other Maldroid 2020 data set. However, in contrast it contains multiple csv files and we need to spend considerable effort to create a data set that can be used for our assignment.

Hence, the Drebin dataset is originally chosen as made it easier to work with for our purpose and demonstrate the ML processes. Also it is well documented or widely used for android malware detection research. There are other papers that describe Drebin data like <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10314496>

However, after running exploratory data analysis, we discovered that Drebin dataset is flawed given it contains a lot of duplicates and we found a paper that highlighted the same issue <https://ieeexplore.ieee.org/document/9609892>

Data Exploration And Analysis

Let's do basic analysis of our dataset and hopefully we gain some insights.

Import Dependencies

```
In [44]: import os
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
```

Load data from our github repository

```
In [45]: url = "https://raw.githubusercontent.com/raymondsamalo/25S1-C-NYP-ITI111-App
if os.path.exists("drebin-215-dataset-5560malware-9476-benign.csv"):
    url = "drebin-215-dataset-5560malware-9476-benign.csv" # if the dataset
df_raw = pd.read_csv(url)
pd.set_option('display.max_columns', None) # read our data frame
print("show size")
print(df_raw.shape)
print("show Info")
df_raw.info()
```

```
show size
(15036, 216)
show Info
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15036 entries, 0 to 15035
Columns: 216 entries, transact to class
dtypes: int64(214), object(2)
memory usage: 24.8+ MB
```

From shape, we found that our dataset has a large number of samples 15036 and large number of features 216.

```
Columns: 216 entries, transact to class
dtypes: int64(214), object(2)
```

From info(), we found that our dataset has 216 columns and 2 of the columns are object while the rest are int64 / numeric.

```
In [46]: df_raw.describe(include=[object]) # show statistics of our data frame
```

```
Out[46]:
```

	TelephonyManager.getSimCountryIso	class
count	15036	15036
unique	5	2
top	0	B
freq	6994	9476

We found that we have 2 object features/columns:

- class
- TelephonyManager.getSimCountryIso

```
In [47]: df_raw.head()
```

```
Out [47]:
```

	transact	onServiceConnected	bindService	attachInterface	ServiceConnection	an
0	0	0	0	0	0	
1	0	0	0	0	0	
2	0	0	0	0	0	
3	0	0	0	0	0	
4	0	0	0	0	0	

```
In [48]: print("Confirm our hypothesis that we have 2 object features/columns and 214 binary columns")
# Display unique values in each column
binary_columns_count = 0
object_columns_count = 0
other_columns_count = 0
total_columns = len(df_raw.columns)
binary_set = {0, 1}
for col in df_raw.columns:
    if df_raw[col].dtype == np.int64:
        if df_raw[col].nunique() == 2 and set(df_raw[col].unique().tolist()) == binary_set:
            binary_columns_count += 1
    elif df_raw[col].dtype == object:
        object_columns_count += 1
    else:
        other_columns_count += 1
print(f"Binary columns: {binary_columns_count} ({(binary_columns_count / total_columns) * 100}%)")
print(f"Object columns: {object_columns_count} ({(object_columns_count / total_columns) * 100}%)")
print(f"Other columns: {other_columns_count} ({(other_columns_count / total_columns) * 100}%)")
```

Confirm our hypothesis that we have 2 object features/columns and 214 binary columns

Binary columns: 214 (99.07%)

Object columns: 2 (0.93%)

Other columns: 0 (0.00%)

From basic analysis we found that our data contains mostly 0 or 1 which represent whether the Application uses the android features/function call or not.

We have quite a number of columns or features.

From head we found column 'class' has our desired output

```
In [49]: df_raw["class"].unique()
```

```
Out [49]: array(['S', 'B'], dtype=object)
```

Class column contains 'S' and 'B' values where S stands for suspicious or malware and B for benign. Let's check whether our data is balanced enough by counting the values.

```
In [50]: print(df_raw['class'].value_counts())
df_raw['class'].value_counts(normalize=True)
```

```

class
B      9476
S      5560
Name: count, dtype: int64

```

```

Out[50]: class
B      0.630221
S      0.369779
Name: proportion, dtype: float64

```

We can see that 63% of data is benign and 36% of data is suspicious or malware. That is sufficiently balanced for our training purpose

Data Cleaning And Preprocessing

Clean Up Column 92 and Object Columns

Pandas reported issue for columns 92 `DtypeWarning: Columns (92) have mixed types. Specify dtype option on import or set low_memory=False.`

Let's take a deeper look on this column 92.

```

In [51]: column_names_index = df_raw.columns
print(column_names_index[92]) # output TelephonyManager.getSimCountryIso
# output array(['0', '1', '?', 1, 0], dtype=object)
print(df_raw['TelephonyManager.getSimCountryIso'].unique())

```

```

TelephonyManager.getSimCountryIso
['0' '1' '?' 1 0]

```

We discovered that the column 92 or `TelephonyManager.getSimCountryIso` contains `['0' '1' '?' 1 0]`.

We need to handle unknown '?' data and also convert '0','1' to integer.

However, the `df.info()` shows us that we have two object columns. Let's check out the other object column to gain deeper insight to our data.

```

In [52]: object_columns = df_raw.select_dtypes(include=['object']).columns.tolist()
# Object columns: ['TelephonyManager.getSimCountryIso', 'class']
print(f"Object columns: {object_columns}")
print(f"{'Column':40}Values")
for i in object_columns:
    print(f"{'i':40}{df_raw[i].unique()}")

```

```

Object columns: ['TelephonyManager.getSimCountryIso', 'class']
Column                                     Values
TelephonyManager.getSimCountryIso         ['0' '1' '?' 1 0]
class                                       ['S' 'B']

```

We discover that beside 'TelephonyManager.getSimCountryIso', the other object or string column is 'class'. For 'class' column, the values are ['S' 'B']. We do not need to handle missing value for 'class' column but we do need to convert 'S' to suspicious malware and 'B' to benign. We shall do this by converting the value to integer 1 for Malware and 0 for benign in a new column.

Alright, given we know the two columns that we need to handle, let's preprocess them.

```
In [53]: get_sim_country_column = 'TelephonyManager.getSimCountryIso'
labelmap={'S':1, 'B':0}
df_raw['malware'] = df_raw['class'].map(labelmap)
df_raw.drop('class', axis=1, inplace=True)
```

We transform column 'class' from 'S', 'B' to column 'malware' with 1,0

```
In [54]: before_values = df_raw[get_sim_country_column].unique()
df_raw[get_sim_country_column] = pd.to_numeric(
    df_raw[get_sim_country_column], errors='coerce')
after_values = df_raw[get_sim_country_column].unique()

print(f"Check values of {get_sim_country_column}")
print(f"before {before_values} -> after {after_values}")

print("Check column values")
all_zeros_or_ones_nan = df_raw.isin([0, 1, np.nan]).all().all()
print(f"is all values are in [0,1, NaN]? {all_zeros_or_ones_nan}")
```

Check values of TelephonyManager.getSimCountryIso

before ['0' '1' '?' '1' 0] -> after [0. 1. nan]

Check column values

is all values are in [0,1, NaN]? True

It seemed our data is simply 0 or 1 or NaN. There is no "out of range" data given our data is technically boolean True = 1 or False = 0

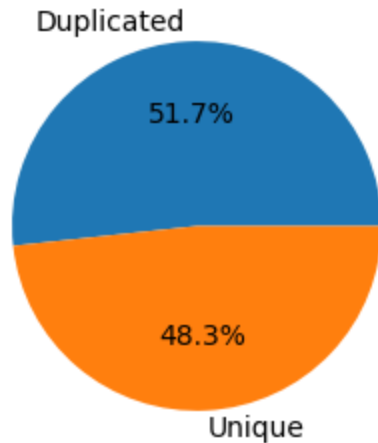
Check For Duplicates Data

First let's check duplicates data.

```
In [55]: duplicated_row_count = df_raw.duplicated().sum()
total_row_count = df_raw.shape[0]
duplicated_row_percentage = (duplicated_row_count/total_row_count*100)
print(f"Total rows count: {total_row_count}")
print(f"Duplicated rows count: {duplicated_row_count}")
print(f"Duplicated rows percentage: {duplicated_row_percentage}")
plt.figure(figsize=(3,3))
sizes = np.array([duplicated_row_count, total_row_count-duplicated_row_count])
labels = ['Duplicated', 'Unique']
plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title("Duplicated Samples in Drebin DataSet")
plt.show()
```

Total rows count: 15036
Duplicated rows count: 7775
Duplicated rows percentage: 51.70923117850492

Duplicated Samples in Drebin DataSet



During this analysis we are surprised that Debrin data has quite a number of duplicates:

Total rows count: 15036
Duplicated rows count: 7775
Duplicated rows percentage: 51.70923117850492

More than half of rows in Debrin is a duplicate, we searched internet and discover a paper that mention the same problem <https://ieeexplore.ieee.org/document/9609892>

We decided to continue with Debrin dataset with the duplicates removed

```
In [56]: def autopct_format(values):
def my_format(pct):
    total = sum(values)
    val = int(round(pct*total/100.0))
    return '{:.1f}%({v:d})'.format(pct, v=val)
return my_format

df_no_duplicates = df_raw.drop_duplicates()
print("Non Duplicated rows:\n", df_no_duplicates.shape[0])
print("Check whether we have sufficient malware vs non malware data")
malware_no_dup=df_no_duplicates["malware"].value_counts()
malware_original=df_raw["malware"].value_counts()
fig = plt.figure(figsize=(10, 5), layout='constrained')
ax = fig.add_subplot(1,2,1)
size = 0.5
vals = np.array([[malware_original[0]-malware_no_dup[0], malware_no_dup[0]],
                 [malware_original[1]-malware_no_dup[1], malware_no_dup[1]]])
print(vals)
tab20c = plt.color_sequences["tab20c"]
outer_colors = [tab20c[i] for i in [0, 4]]
inner_colors = [tab20c[i] for i in [19, 0,19,4]]
bm_data=vals.sum(axis=1)
ax.pie(bm_data, radius=1, colors=outer_colors, autopct=autopct_format(bm_data))
```



```

        wedgeprops=dict(width=size, edgecolor='w'), labels=["Benign", "Malware"])
    bm_data=vals.flatten()
    ax.pie(bm_data, radius=1-size, colors=inner_colors,
           wedgeprops=dict(width=size, edgecolor='w'),
           labeldistance=0.7, labels=["Duplicates","Unique", "Duplicates","Unique"])
    ax.set(aspect="equal", title='Benign vs Malware \n With Duplicated Samples')
    ax = fig.add_subplot(1,2,2)
    ax.pie(malware_no_dup, colors=outer_colors,
           wedgeprops=dict(width=size, edgecolor='w'),
           labels=["Benign", "Malware"], autopct=autopct_format(malware_no_dup))
    ax.set(aspect="equal", title='Benign vs Malware \n After Duplicates Removal')
    plt.show()

```

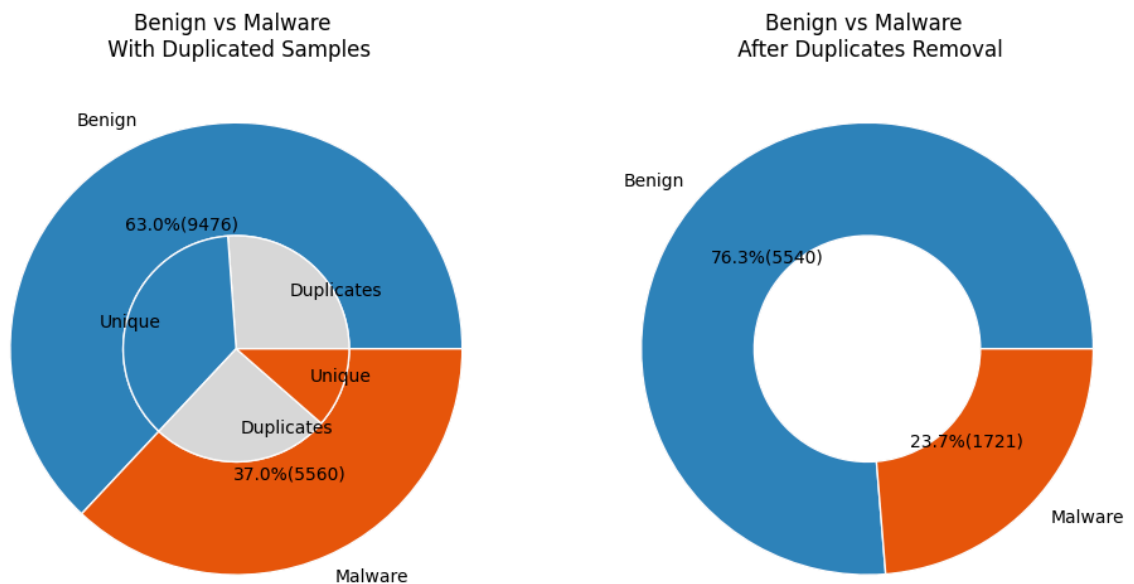
Non Duplicated rows:

7261

Check whether we have sufficient malware vs non malware data

[[3936 5540]

[3839 1721]]



After removing the duplicate rows, our data set is still reasonably sufficient and balanced for our need with

- benign sample 5540
- malware sample 1721 Roughly 23 % data shows malware and 76 % benign.

Let's deal with missing data next.

```
In [57]: df = df_no_duplicates.copy() # replace our data frame with non-duplicated r
```

Handle Missing Data

```
In [58]: print("List all column with missing data")
# return ['TelephonyManager.getSimCountryIso'] as the only column with nan
print(df.columns[df.isna().any()].tolist())
rows_with_nan = df[df.isnull().any(axis=1)].index
```

```
no_of_rows = df.shape[0]
print("List all rows with missing data")
print(rows_with_nan)
no_of_rows_missing_data = rows_with_nan.shape[0]
print(f"Missing data is {no_of_rows_missing_data} out of {no_of_rows}")
```

```
List all column with missing data
['TelephonyManager.getSimCountryIso']
List all rows with missing data
Index([176, 2109], dtype='int64')
Missing data is 2 out of 7261
```

Given we only have 2 rows out of 7261 with missing data, let us simply remove them.

```
In [59]: df.dropna(inplace=True)
rows_with_nan = df[df.isnull().any(axis=1)].index
no_of_rows_missing_data = rows_with_nan.shape[0]
no_of_rows = df.shape[0]
print(
    f"No of rows with missing data is now {no_of_rows_missing_data} out of {
```

```
No of rows with missing data is now 0 out of 7259
```

Split Feature vs Labels

```
In [60]: malware_index = df.columns.get_loc("malware")
print("Index of 'malware' column:", malware_index)
y = df['malware']
X = df.loc[:, df.columns != 'malware']
```

```
Index of 'malware' column: 215
```

Is Our DataSet Imbalanced ?

According to <https://developers.google.com/machine-learning/crash-course/overfitting/imbalanced-datasets>

We do have imbalanced dataset but it is a mild imbalanced set (20-40% of the dataset is minority case).

This is true for our dataset before and after duplicate removal

Given it is a mild case, we should first train our model using the dataset before we attempt to do any attempt to rebalance the dataset.

However, let's create an oversampled data set just in case

```
In [61]: %pip install imblearn
# Check if the imbalanced-learn package is installed
try:
    from imblearn.over_sampling import SMOTE
    print("imblearn package is installed.")
```

```

oversample = SMOTE()
over_X, over_y = oversample.fit_resample(X, y)
print("Oversampling completed.")
print(f"Original class distribution:\n{y.value_counts()}")
print(f"Oversampled class distribution:\n{over_y.value_counts()}")

plot_df = pd.DataFrame({
    'Original': y.value_counts(),
    'Oversampled': over_y.value_counts()
}).T
plot_df.plot(kind='bar', stacked=True, rot=0)
plt.title('Stacked Value Counts of Binary Columns')
plt.ylabel('Count')
plt.xlabel('Binary Value')
plt.show()
except ImportError:
    print("imblearn package is not installed. Please install it using 'pip i

```

Requirement already satisfied: imblearn in /Users/Ray/.local/lib/python3.13/site-packages (0.0)

Requirement already satisfied: imbalanced-learn in /Users/Ray/.local/lib/python3.13/site-packages (from imblearn) (0.13.0)

Requirement already satisfied: numpy<3,>=1.24.3 in /opt/anaconda3/envs/ml/lib/python3.13/site-packages (from imbalanced-learn->imblearn) (2.2.5)

Requirement already satisfied: scipy<2,>=1.10.1 in /opt/anaconda3/envs/ml/lib/python3.13/site-packages (from imbalanced-learn->imblearn) (1.15.2)

Requirement already satisfied: scikit-learn<2,>=1.3.2 in /opt/anaconda3/envs/ml/lib/python3.13/site-packages (from imbalanced-learn->imblearn) (1.6.1)

Requirement already satisfied: sklearn-compat<1,>=0.1 in /Users/Ray/.local/lib/python3.13/site-packages (from imbalanced-learn->imblearn) (0.1.3)

Requirement already satisfied: joblib<2,>=1.1.1 in /opt/anaconda3/envs/ml/lib/python3.13/site-packages (from imbalanced-learn->imblearn) (1.5.0)

Requirement already satisfied: threadpoolctl<4,>=2.0.0 in /opt/anaconda3/envs/ml/lib/python3.13/site-packages (from imbalanced-learn->imblearn) (3.6.0)

Note: you may need to restart the kernel to use updated packages.

imblearn package is installed.

Oversampling completed.

Original class distribution:

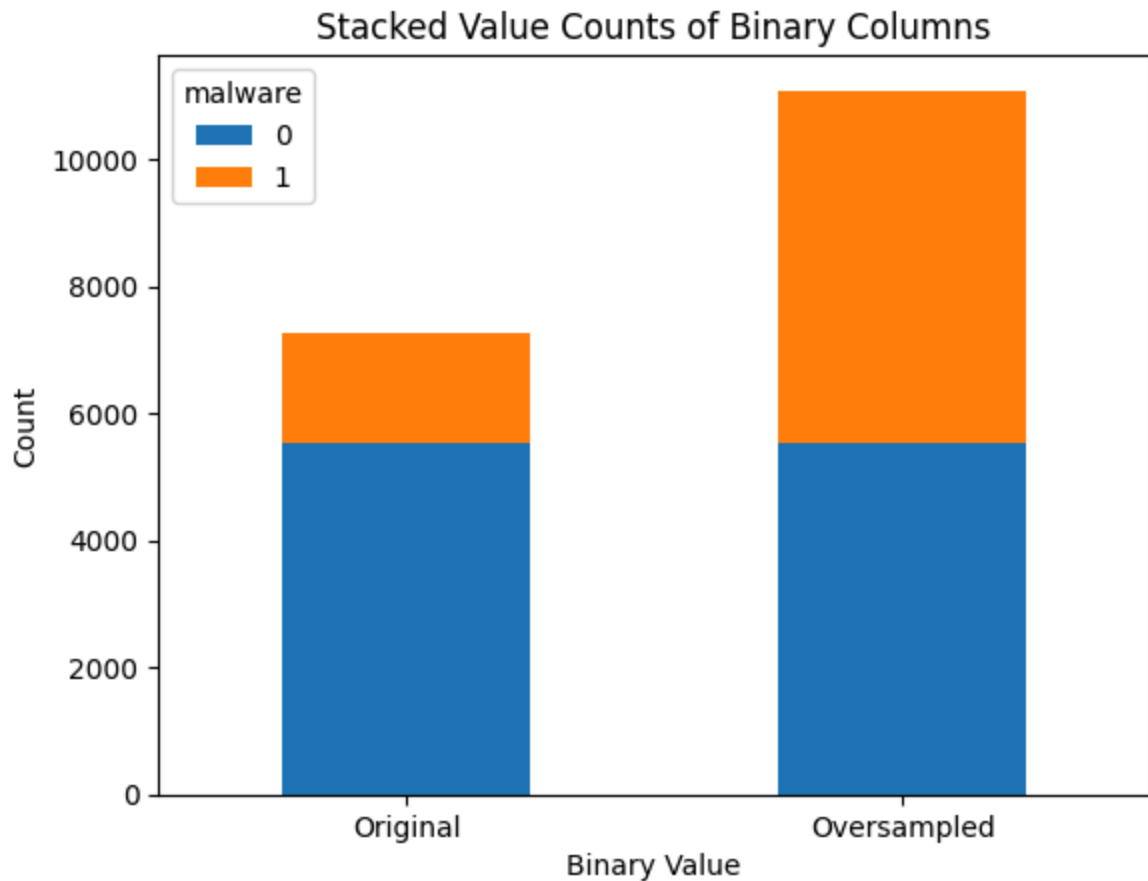
malware	
0	5540
1	1719

Name: count, dtype: int64

Oversampled class distribution:

malware	
1	5540
0	5540

Name: count, dtype: int64



Splitting

Let's split our data into test and train rows

```
In [62]: # Get index of the "malware" column
from sklearn.model_selection import train_test_split
# we use our oversampled data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, stratify=y, random_state=42)
over_X_train, over_X_test, over_y_train, over_y_test = train_test_split(
    over_X, over_y, stratify=over_y, random_state=42)

print(X_train.shape)
print(X_test.shape)
print(over_X_train.shape)
print(over_X_test.shape)
```

```
(5444, 215)
```

```
(1815, 215)
```

```
(8310, 215)
```

```
(2770, 215)
```

Let's check if the data is stratified properly

```
In [63]: print("Train Data Set Output")
print(y_train.value_counts())
print(y_train.value_counts(normalize=True))
```

```

print("Test Data Set Output")
print(y_test.value_counts())
print(y_test.value_counts(normalize=True))
print("Oversampled Train Data Set Output")
print(over_y_train.value_counts())
print(over_y_train.value_counts(normalize=True))
print("Oversampled Test Data Set Output")
print(over_y_test.value_counts())
print(over_y_test.value_counts(normalize=True))

```

Train Data Set Output

malware

0 4155

1 1289

Name: count, dtype: int64

malware

0 0.763226

1 0.236774

Name: proportion, dtype: float64

Test Data Set Output

malware

0 1385

1 430

Name: count, dtype: int64

malware

0 0.763085

1 0.236915

Name: proportion, dtype: float64

Oversampled Train Data Set Output

malware

0 4155

1 4155

Name: count, dtype: int64

malware

0 0.5

1 0.5

Name: proportion, dtype: float64

Oversampled Test Data Set Output

malware

0 1385

1 1385

Name: count, dtype: int64

malware

0 0.5

1 0.5

Name: proportion, dtype: float64

Looks like our data is stratified and split properly

Correlation Matrix

It is common to use correlation matrix for identifying feature that is redundant and can be dropped or to see relationship between features

However, given we have 215 features, it is hard to do this using visual inspection like heatmap.

Hence, we shall do a bit of code to automate analysis of correlation matrix.

```
In [64]: correlation_matrix = X_train.corr()
# set our threshold -> 1 highly correlated .. 0 not correlated
threshold = 0.8
# form pairs and sort from highest to lowest ( we want absolute value )
corr_pairs = correlation_matrix.abs().unstack().sort_values(kind="quicksort")
features_corrs: list[set]=[] # list of set that grow for each pair that is hi
for a, b in corr_pairs.index: # for each pair of features in the correlatio
    # if a and b are not the same feature and the correlation is above the t
    if a!=b and corr_pairs[(a,b)] > threshold:
        new_feature_set=None
        found = False
        for feature_set in features_corrs: # for each existing set of featur
            if a in feature_set or b in feature_set: # if either a or b is
                # expand existing set with a and b
                # set will not contain duplicate so we can add the same feat
                feature_set.add(b) # add to existing set
                feature_set.add(a) # add to existing set
                found = True
            break
        # if there is no existing set that contains a or b
        # then we create a new set with a and b
        if not found and a!=b: # no existing set exist
            features_corrs.append(set([a,b])) # add new set
# we will count how many features we have left if we remove highly correlat
feature_count=(X_train.shape[1])
for feature_set in features_corrs:
    feature_count=feature_count-len(feature_set)+1 # count feature left if w
    print(feature_set)
print("final count ", feature_count)
```

```
{'IBinder', 'android.os.IBinder', 'Binder'}
{'onServiceConnected', 'ServiceConnection', 'bindService', 'android.os.Binder'}
{'SUBSCRIBED_FEEDS_READ', 'SUBSCRIBED_FEEDS_WRITE'}
{'android.os.Binder', 'transact', 'attachInterface'}
{'Ljavax.crypto.spec.SecretKeySpec', 'SecretKey', 'Ljavax.crypto.Cipher', 'KeySpec'}
{'WRITE_HISTORY_BOOKMARKS', 'READ_HISTORY_BOOKMARKS'}
{'READ_SYNC_SETTINGS', 'WRITE_SYNC_SETTINGS'}
{'WRITE_USER_DICTIONARY', 'READ_USER_DICTIONARY'}
{'android.intent.action.SCREEN_ON', 'android.intent.action.SCREEN_OFF'}
{'RECEIVE_BOOT_COMPLETED', 'android.intent.action.BOOT_COMPLETED'}
{'Landroid.content.Context.registerReceiver', 'Landroid.content.Context.unregisterReceiver'}
{'WRITE_SOCIAL_STREAM', 'READ_SOCIAL_STREAM'}
{'WRITE_CALENDAR', 'READ_CALENDAR'}
{'HttpPost.init', 'HttpRequest'}
final count 195
```

The highly correlated sets look reasonable e.g

```
{'android.intent.action.BOOT_COMPLETED', 'RECEIVE_BOOT_COMPLETED'}
```

indicates that the app need to listen to notification by android OS that boot completed and has ability to received the notification.

Let's build a training data set with highly correlated feature remove, just in case we need it

```
In [65]: all_columns = set(X_train.columns.to_list())
columns_to_drop=set()
for feature_set in features_corrs:
    feature_set = sorted(list(feature_set)) # prioritise read over write as
    for feature in feature_set[1:]: # keep first feature
        columns_to_drop.add(feature)
columns_to_keep = all_columns - columns_to_drop
# we create both training and test data set with highly correlated feature r
X_train_cfs = X_train.filter(items=list(columns_to_keep))
X_test_cfs = X_test.filter(items=list(columns_to_keep))
print("number of column after dropping highly correlated columns",X_train_cfs.shape[1])
print("number of column before dropping highly correlated columns",X_train.columns.shape[1])
print("Percentage of features dropped: {:.2f}%".format((1 - X_train_cfs.shape[1] / X_train.columns.shape[1])))
```

```
number of column after dropping highly correlated columns 195
number of column before dropping highly correlated columns 215
Percentage of features dropped: 9.30%
```

While it look promising, it is also quite risky to drop the features as we do not have sufficient domain knowledge to know which one to drop

Two Classical ML Models

We would like to compare the accuracy of various classification machine learning algorithms and pick the best 2 models.

Some classification algorithm requires us to tune the hyper parameters to get the best performance e.g Decision Tree.

Classical ML Comparison

Let's compare some well known classical ML Methods binary classifiers and pick 2 best classifiers that fit our dataset

Our candidates are :

- Logistic Regression which is commonly used for classification
- Decision Tree
- Random Forest which is a collection of Decision Tree

- Bernouli Naive Bayes which is designed specifically for binary features and binary label similar to our dataset
- K-Nearest Neighbour

We also try to detect overfitting automatically using the following indicators :

- If accuracy on training data set differ from test data set accuracy by 5%, we consider it significant enough to suspect overfitting.
- If accuracy on training data set is perfect i.e 1.0 or 100% then we suspect overfitting
-> of course we need to check further

```
In [66]: from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import BernoulliNB
from sklearn.neighbors import KNeighborsClassifier
# Define classifiers and run comparison
def create_classifiers():
    # Create classifiers
    # we do this so we can reuse the same classifiers for both original and
    lr_clf = LogisticRegression(random_state=42)
    dt_clf = DecisionTreeClassifier(random_state=42)
    rf_clf = RandomForestClassifier(random_state=42)
    bnb_clf = BernoulliNB() # designed for binary features
    knn_clf = KNeighborsClassifier(n_neighbors=5) # KNN with 5 neighbors
    classifiers = {
        'Bernoulli Naive Bayes': bnb_clf,
        'Decision Tree': dt_clf,
        'Logistic Regression': lr_clf,
        'Random Forest': rf_clf,
        'K-Nearest Neighbors': knn_clf
    }
    return classifiers
```

```
In [67]: from sklearn.model_selection import cross_val_score
from sklearn.model_selection import StratifiedKFold

# we create a function to evaluate the classifier so that we can reuse it
# in evaluating different classifiers
# for both classical ML and Ensemble Methods
def evaluate_classifier(clf, train_X, train_y, test_X, test_y):
    """ Evaluate the classifier using cross-validation and return various me
    Args:
        clf: The classifier to evaluate.
        train_X: Training features.
        train_y: Training labels.
        test_X: Testing features.
        test_y: Testing labels.
    """
    # Create a stratified k-fold cross-validator given our data is imbalance
    stratified_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=4
    train_scores = cross_val_score(clf, train_X, train_y, cv=stratified_cv,
```



```

cv_accuracy_mean = train_scores.mean()
clf.fit(train_X, train_y) # fit the classifier on the training data
train_accuracy = clf.score(train_X, train_y)
test_accuracy = clf.score(test_X, test_y)
delta_accuracy = train_accuracy - test_accuracy # difference between tra
cv_delta_accuracy= cv_accuracy_mean - test_accuracy # difference between
delta_accuracy_overfitting = (delta_accuracy > 0.05) # threshold for ove
train_accuracy_overfitting = (train_accuracy >= 1.0) # if train accuracy
overfitting = (delta_accuracy_overfitting or train_accuracy_overfitting)
return {
    'cv_accuracy_mean': cv_accuracy_mean,
    'train_accuracy': train_accuracy,
    'test_accuracy': test_accuracy,
    'delta_accuracy': delta_accuracy,
    'cv_delta_accuracy': cv_delta_accuracy,
    'overfitting': overfitting
}

def compare_classifiers(train_X, train_y, test_X, test_y, classifiers, pick)
best_clfs = []
clfs_dict = {}
# evaluate each classifier and store the results in a dictionary
for label, clf in classifiers.items():
    clfs_dict[label] = evaluate_classifier(clf, train_X, train_y, test_X)
# sort classifiers by test accuracy
for label, value in sorted(clfs_dict.items(), key=lambda x: x[1]['cv_acc
print(f"{label:30} ",
      f" CV Mean Acc: {value['cv_accuracy_mean']:.5f}",
      f" Test Acc: {value['test_accuracy']:.5f}",
      f" Train Acc: {value['train_accuracy']:.5f}",
      f" Possible Overfitting: {str(value['overfitting']):5}",
      f" Delta : {value['delta_accuracy']:.5f}",
      f" CV Delta : {value['cv_delta_accuracy']:.5f}"
    )
    # pick the best classifiers based on test accuracy
    if len(best_clfs) < pick:
        best_clfs.append(label)
print(f"\n Best {pick} classifiers: {' '.join(best_clfs)}")

```

```

In [68]: print("Compare classifiers on original data set")
print("=====")
compare_classifiers(X_train, y_train, X_test, y_test, create_classifiers(),

```

Compare classifiers on original data set

```
=====
Random Forest                CV Mean Acc: 0.96675  Test Acc: 0.97190  Tr
ain Acc: 1.00000  Possible Overfitting: True  Delta : 0.02810  CV Delta : -
0.00515
Logistic Regression          CV Mean Acc: 0.96528  Test Acc: 0.96860  Tr
ain Acc: 0.97520  Possible Overfitting: False  Delta : 0.00661  CV Delta : -
0.00331
K-Nearest Neighbors          CV Mean Acc: 0.96051  Test Acc: 0.96529  Tr
ain Acc: 0.97832  Possible Overfitting: False  Delta : 0.01304  CV Delta : -
0.00478
Decision Tree                CV Mean Acc: 0.92965  Test Acc: 0.93609  Tr
ain Acc: 1.00000  Possible Overfitting: True  Delta : 0.06391  CV Delta : -
0.00644
Bernoulli Naive Bayes        CV Mean Acc: 0.82072  Test Acc: 0.82369  Tr
ain Acc: 0.82090  Possible Overfitting: False  Delta : -0.00279  CV Delta :
-0.00297
```

Best 2 classifiers: Random Forest, Logistic Regression

We found that the two best classifiers based on cross validation accuracy are 'Random Forest' and 'Logistic Regression'

We also do a quick check of overfitting between Test Data and Training Data accuracy score.

Overfitting

If the difference between Train and Test accuracy is > 0.05 then it is an indicator of overfitting.

Here the Decision Tree classifier exhibit this case.

Also if train accuracy has no error i.e. perfect prediction then it is possible overfitting

Hence, the random forest may also exhibit overfitting.

This could be caused by our imbalanced dataset (<https://medium.com/sfu-csmp/surviving-in-a-random-forest-with-imbalanced-datasets-b98b963d52eb>)

Let's check what happened if we use our oversampled balanced dataset instead

Compare classifier on balanced dataset

```
In [69]: print("Compare classifiers on oversampled data set")
print("=====")
compare_classifiers(over_X_train, over_y_train, over_X_test, over_y_test, cr
```

Compare classifiers on oversampled data set

```
=====
Random Forest                CV Mean Acc: 0.97256  Test Acc: 0.97834  Train Acc: 0.99904  Possible Overfitting: False  Delta : 0.02070  CV Delta : -0.00578
Logistic Regression          CV Mean Acc: 0.96161  Test Acc: 0.96318  Train Acc: 0.96763  Possible Overfitting: False  Delta : 0.00445  CV Delta : -0.00156
Decision Tree                 CV Mean Acc: 0.94609  Test Acc: 0.95271  Train Acc: 0.99904  Possible Overfitting: False  Delta : 0.04633  CV Delta : -0.00662
K-Nearest Neighbors           CV Mean Acc: 0.94019  Test Acc: 0.94801  Train Acc: 0.96450  Possible Overfitting: False  Delta : 0.01649  CV Delta : -0.00782
Bernoulli Naive Bayes         CV Mean Acc: 0.85499  Test Acc: 0.86137  Train Acc: 0.85656  Possible Overfitting: False  Delta : -0.00481  CV Delta : -0.00638
```

Best 2 classifiers: Random Forest, Logistic Regression

This is much better than original data.

Decision Tree still show overfitting but we are not going to choose this as it does not perform as good as Random Forest and Logistic Regression anyway.

Let's see if we can tune hyperparameters for our Random Forest and Logistic Regression

Random Forest

Let's evaluate our Random Forest classifier performance and see whether we could tune its hyperparameter

```
In [70]: from sklearn.model_selection import GridSearchCV
import warnings
warnings.filterwarnings("ignore")

# Set rule to ignore warnings
param_grid = {
    'max_features': ['sqrt', 'log2'],
    'max_depth' : [4, 6, 8, 10, 12, None],
    'n_estimators': [1, 8, 16, 32, 64, 100],
}
rf_clf = RandomForestClassifier(random_state=42)
rf_clf.fit(over_X_train, over_y_train) # fit the classifier on the training
clf = GridSearchCV(rf_clf, param_grid = param_grid, cv = 3, verbose=False)
best_rf_clf = clf.fit(over_X_train, over_y_train)
best_rf_clf.best_estimator_
print("Best parameters for Random Forest Regression:")
print(best_rf_clf.best_params_)
print("Best score for Random Forest Regression:")
print(best_rf_clf.best_score_)
```

Best parameters for Random Forest Regression:
{ 'max_depth': None, 'max_features': 'log2', 'n_estimators': 100}
Best score for Random Forest Regression:
0.9693140794223827

```
In [71]: def create_tuned_random_forest_classifier():  
        return RandomForestClassifier(  
            max_depth=best_rf_clf.best_params_["max_depth"],  
            max_features=best_rf_clf.best_params_["max_features"],  
            n_estimators=best_rf_clf.best_params_["n_estimators"],  
            random_state=42  
        )
```

The grid search found that the best hyperparameters and we used in our create_tuned_random_forest_classifier above.

Now we check with un-tuned Random Forest Classifier and compare

```
In [72]: classifiers = {  
        'Random Forest Regression': RandomForestClassifier(random_state=42),  
        'Random Forest Regression Tuned': create_tuned_random_forest_classifier()  
    }  
compare_classifiers(over_X_train, over_y_train, over_X_test, over_y_test, classifiers)
```

Random Forest Regression	CV Mean Acc: 0.97256	Test Acc: 0.97834	Train Acc: 0.99904	Possible Overfitting: False	Delta : 0.02070	CV Delta : -0.00578
Random Forest Regression Tuned	CV Mean Acc: 0.97244	Test Acc: 0.97978	Train Acc: 0.99904	Possible Overfitting: False	Delta : 0.01925	CV Delta : -0.00734

Best 1 classifiers: Random Forest Regression

Logistic Regression

Let's try to do hyper parameter tuning for our logistic regression

```
In [81]: import warnings  
        from sklearn.model_selection import GridSearchCV  
  
        warnings.filterwarnings("ignore")  
  
        # Set rule to ignore warnings  
        param_grid = [  
            {'penalty': ['l1', 'l2', 'elasticnet', 'none'],  
             'solver': ['lbfgs', 'newton-cg', 'liblinear', 'sag', 'saga'],  
             'C' : [0.001, 0.01, 0.1, 1, 10, 100]}, # regularization strength  
        ]  
  
        lr_clf = LogisticRegression(random_state=42)  
        lr_clf.fit(over_X_train, over_y_train) # fit the classifier on the training data  
        clf = GridSearchCV(lr_clf, param_grid = param_grid, cv = 3, verbose=False)  
        best_lr_clf = clf.fit(over_X_train, over_y_train)  
        best_lr_clf.best_estimator_
```

```
print("Best parameters for Logistic Regression:")
print(best_lr_clf.best_params_)
print("Best score for Logistic Regression:")
print(best_lr_clf.best_score_)
```

Best parameters for Logistic Regression:
{'C': 10, 'penalty': 'l2', 'solver': 'saga'}
Best score for Logistic Regression:
0.9611311672683515

NOTE: running this notebook on Google Colab vs locally on laptop may result on different best parameters for Logistic Regression

Hence, we decided to pass the hyper parameters to our method instead to ensure the best parameters is always chosen regardless of the run time environment

Let's compare it with previous logistic regression before we do tuning of hyperparameter

```
In [74]: def create_tuned_logistic_regression_classifier():
            return LogisticRegression(random_state=42,
                                      C=best_lr_clf.best_params_["C"],
                                      solver=best_lr_clf.best_params_["solver"],
                                      penalty=best_lr_clf.best_params_["penalty"])

classifiers = {
    'Logistic Regression': LogisticRegression(random_state=42),
    'Logistic Regression Tuned': create_tuned_logistic_regression_classifier()
}
compare_classifiers(over_X_train, over_y_train, over_X_test, over_y_test, classifiers)
```

Logistic Regression Tuned	CV Mean Acc: 0.96318	Test Acc: 0.96209	Train Acc: 0.96943	Possible Overfitting: False	Delta : 0.00734	CV Delta : 0.00108
Logistic Regression	CV Mean Acc: 0.96161	Test Acc: 0.96318	Train Acc: 0.96763	Possible Overfitting: False	Delta : 0.00445	CV Delta : -0.00156

Best 1 classifiers: Logistic Regression Tuned

With Tuned Logistic regression we have slightly improvement

Voting and Stacking

We also would like to compare ensemble methods like Stacking and Voting algorithm beside simpler ML algorithm.

```
In [75]: from sklearn.ensemble import VotingClassifier, StackingClassifier
            from sklearn.linear_model import LogisticRegression
            from sklearn.ensemble import RandomForestClassifier

lr_clf = create_tuned_logistic_regression_classifier() # Use best parameters
rf_clf = create_tuned_random_forest_classifier() # Set max_depth to avoid
rf_clf_untuned = RandomForestClassifier(random_state=42) # Un-tuned Random F
```

```

stacking_estimators = [ # we need 3 classifiers for voting ensemble methods
    ('lr', lr_clf),
    ('rf', rf_clf),
]
voting_estimators = [ # we need 3 classifiers for voting ensemble methods
    ('lr', lr_clf),
    ('rf', rf_clf),
    ('rf_untuned', rf_clf_untuned), # Add another to provide tie-breaker ca
]

v_h_clf = VotingClassifier(estimators=voting_estimators,voting='hard')
v_s_clf = VotingClassifier(estimators=voting_estimators,voting='soft')
st_clf = StackingClassifier(estimators=stacking_estimators,
                           final_estimator=LogisticRegression(random_state=

classifiers = {
    'Logistic Regression': lr_clf,
    'Random Forest': rf_clf,
    'Voting Hard Ensemble': v_h_clf,
    'Voting Soft Ensemble': v_s_clf,
    'Stacking Ensemble': st_clf,
}
best_model = None
best_model_name = None
clfs_dict = {}
# Compare classifiers and store results
# use our reduced feature set
for label, clf in classifiers.items():
    clfs_dict[label] = evaluate_classifier(clf, over_X_train, over_y_train,
    clfs_dict[label]['clf'] = clf # Store the classifier object
# Print results
for label, value in sorted(clfs_dict.items(), key=lambda x: (x[1]['cv_accuracy_mean']),
    print(f"{label:30} ",
        f" CV Mean: {value['cv_accuracy_mean']:.4f}",
        f" Train Acc: {value['train_accuracy']:.4f}",
        f" Test Accuracy: {value['test_accuracy']:.4f}",
        f" Possible Overfitting: {value['overfitting']}",
        f" Delta : {value['delta_accuracy']:.3f}",
        f" CV Delta : {value['cv_delta_accuracy']:.3f}",
    )
    if best_model is None:
        best_model = value
        best_model_name = label

```

Stacking Ensemble CV Mean: 0.9759 Train Acc: 0.9983 Test Accuracy: 0.9809 Possible Overfitting: False Delta : 0.017 CV Delta : -0.005

Voting Soft Ensemble CV Mean: 0.9758 Train Acc: 0.9976 Test Accuracy: 0.9773 Possible Overfitting: False Delta : 0.020 CV Delta : -0.001

Voting Hard Ensemble CV Mean: 0.9735 Train Acc: 0.9990 Test Accuracy: 0.9801 Possible Overfitting: False Delta : 0.019 CV Delta : -0.007

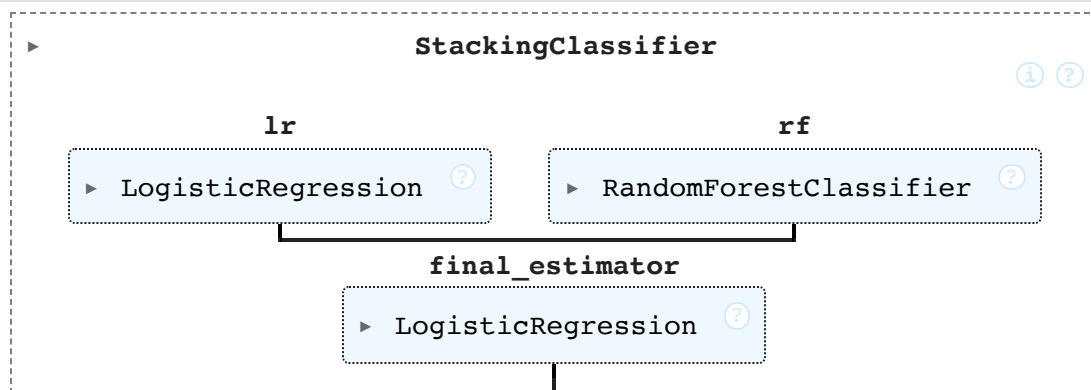
Random Forest CV Mean: 0.9724 Train Acc: 0.9990 Test Accuracy: 0.9798 Possible Overfitting: False Delta : 0.019 CV Delta : -0.007

Logistic Regression CV Mean: 0.9632 Train Acc: 0.9694 Test Accuracy: 0.9621 Possible Overfitting: False Delta : 0.007 CV Delta : 0.001

Our best classifier is Stacking Ensemble based on our training data (cross validation mean accuracy), followed by Soft voting Ensemble

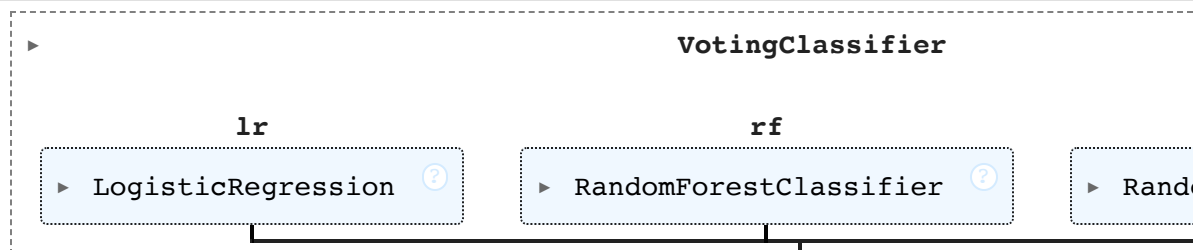
In [76]: `st_clf`

Out[76]:



In [79]: `v_s_clf`

Out[79]:



Performance Evaluation

Let us check the performance of our classifiers using confusion matrix and report

```

In [80]: from sklearn.model_selection import cross_val_predict
         from sklearn.metrics import ConfusionMatrixDisplay

         def display_confusion_matrix(ax, the_lr_clf, train_X, train_y, title):
             pred = cross_val_predict(the_lr_clf, train_X, train_y, cv=5)
             disp=ConfusionMatrixDisplay.from_predictions(train_y, pred,display_label

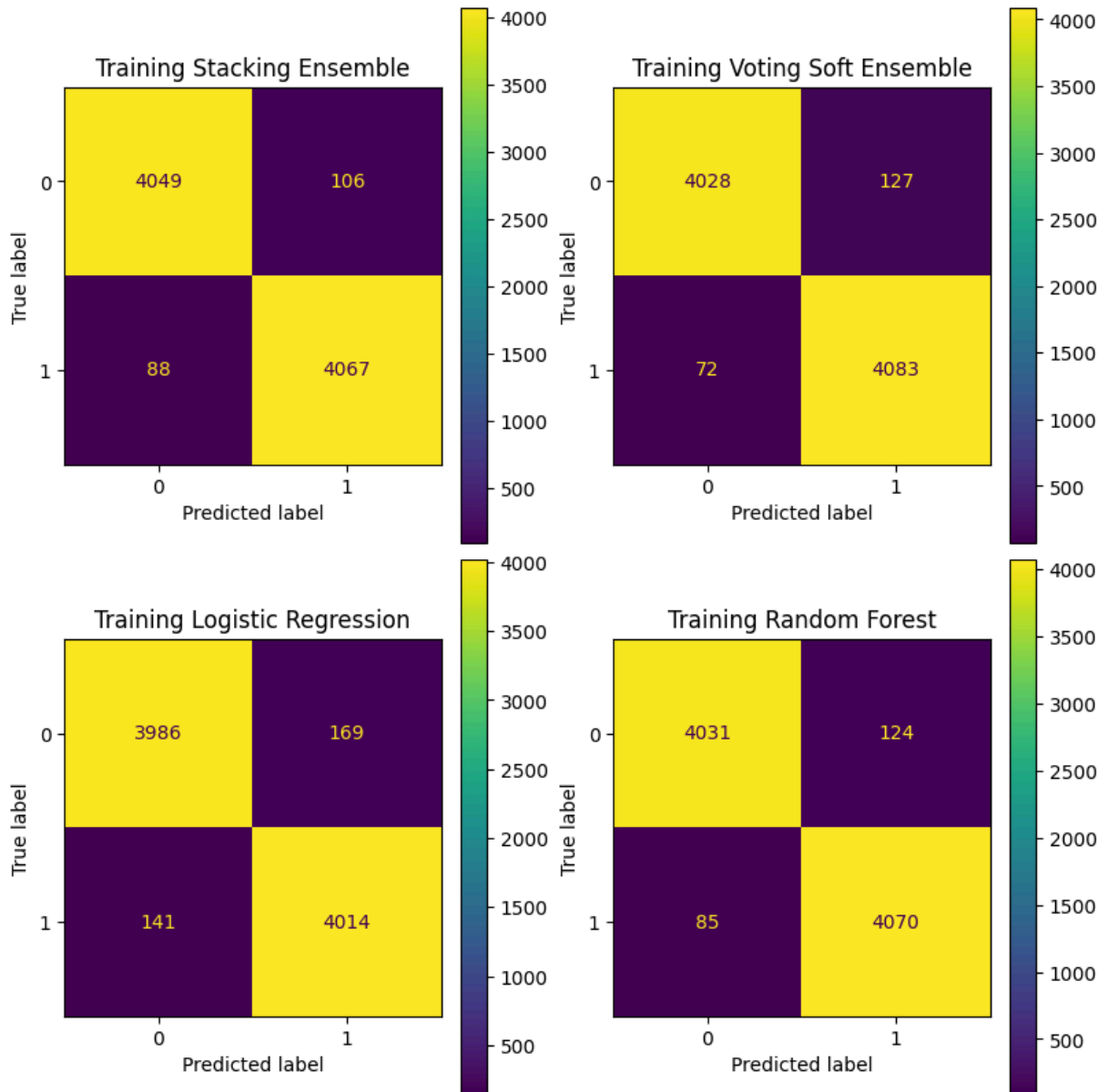
```

```

disp.ax_.set_title(title)

fig = plt.figure(figsize=(8, 8), layout='constrained')
display_confusion_matrix(fig.add_subplot(2,2,1), st_clf, over_X_train, over
display_confusion_matrix(fig.add_subplot(2,2,2), v_s_clf, over_X_train, over
display_confusion_matrix(fig.add_subplot(2,2,3), lr_clf, over_X_train, over
display_confusion_matrix(fig.add_subplot(2,2,4), rf_clf, over_X_train, over

```



For training data, our ensemble methods perform better than Logistic Regression.

However, they do not perform better than Random Forest except for Soft Voting on FN

Finally, let us test with our test samples. Here we simply test with our original test first instead of over sampled test

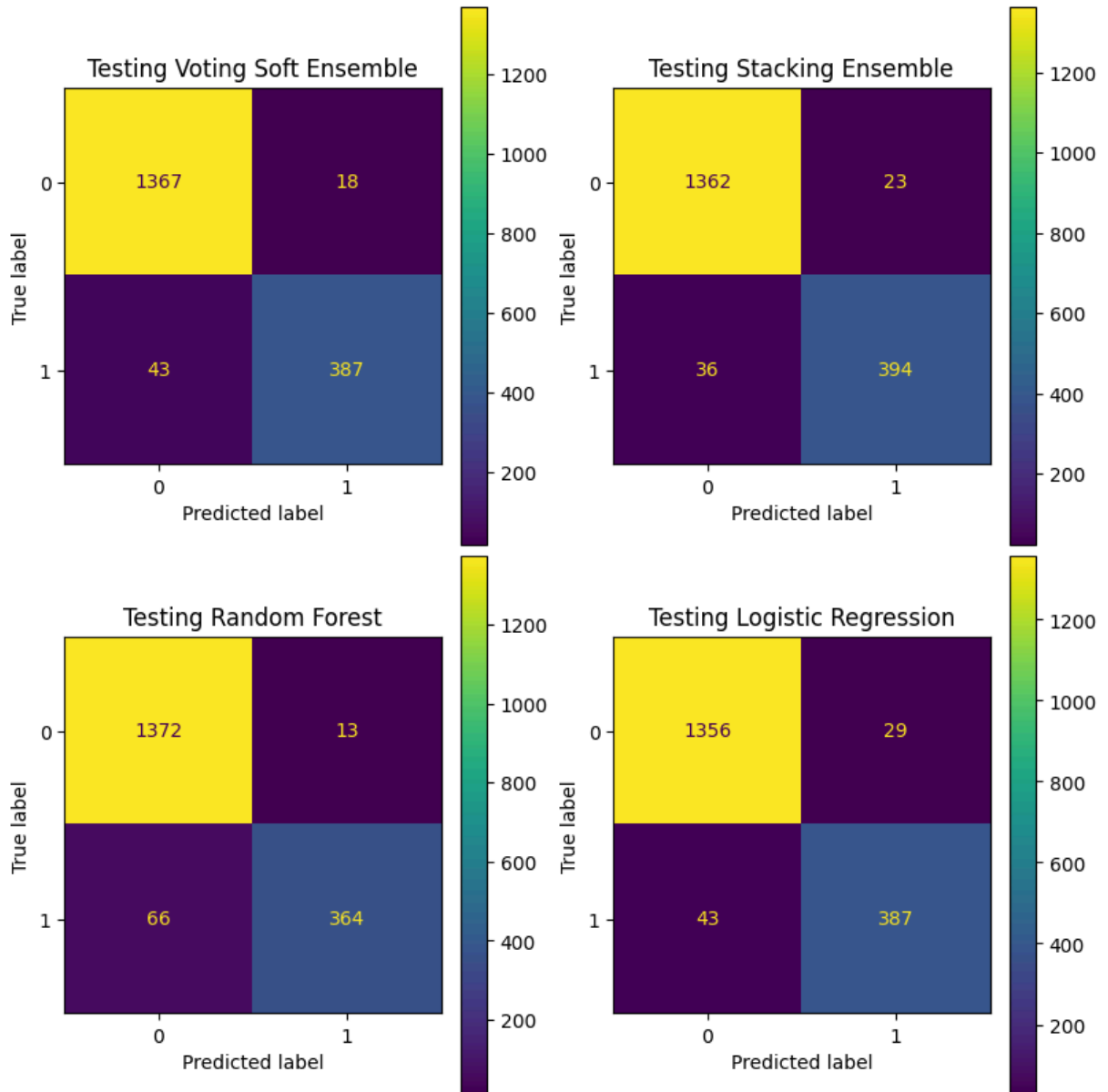
```

In [ ]: fig = plt.figure(figsize=(8, 8), layout='constrained')
display_confusion_matrix(fig.add_subplot(2,2,1), v_s_clf, X_test, y_test, "T
display_confusion_matrix(fig.add_subplot(2,2,2), clf, X_test, y_test, "Testi

```



```
display_confusion_matrix(fig.add_subplot(2,2,3), rf_clf, X_test, y_test, "Testing Voting Soft Ensemble")
display_confusion_matrix(fig.add_subplot(2,2,4), lr_clf, X_test, y_test, "Testing Logistic Regression")
```



For testing data:

- False Negative for Stacking Ensemble is the smallest
- False Positive for Stacking Ensemble is not as good as Random Forest
- Random Forest has lowest False Positive but highest False Negative

Given we try to prevent malware, it is better to have smallest False Negative than False Positive.

- False Positive may cause benign application categorised as malware, it is not ideal but can improve our model by training.
- False Negative may cause a malware to be allowed to damage the system and cause harm.

Hence, the stacking ensemble performed best

```
In [ ]: from sklearn.model_selection import cross_val_predict
        from sklearn.metrics import classification_report

        def print_classification_report(the_lr_clf, train_X, train_y, title):
            pred = cross_val_predict(the_lr_clf, train_X, train_y, cv=5)
            print(title)
            print(classification_report(train_y, pred, digits=4))

        print_classification_report(clf, over_X_train, over_y_train, "Training Stack
```

Training Stacking Ensemble				
	precision	recall	f1-score	support
0	0.9773	0.9723	0.9748	4155
1	0.9725	0.9774	0.9749	4155
accuracy			0.9748	8310
macro avg	0.9749	0.9748	0.9748	8310
weighted avg	0.9749	0.9748	0.9748	8310

Our F1-Score, precision and recall is high for stacking ensemble

Conclusion

We discovered that in our study:

- the preprocessing steps are highly important as we discover duplicate samples that may affect our result.
- imbalance data affect our training and cause overfitting
- ensemble method does have the potential to improve performance of machine learning